SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr DUMPALA PRASANTH

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Analytical Problem-Solving Assessment

Duration: 1hr

1. A warehouse robot must determine the shortest path to transport goods while avoiding obstacles. Apply **Dynamic Programming** to develop an optimal policy and explain how value iteration can improve navigation efficiency**.** **(5 Marks)**

2. A recommendation system collects customer interactions over multiple sessions. Apply **Monte Carlo Control** to update the policy and explain how the agent improves recommendations with experience. **.** **(5 Marks)**

3. A delivery robot operates in an uncertain environment. Apply the **SARSA** algorithm to update the action-value function and explain how the learned policy adapts to environmental changes. **.**
(5 Marks)

4. A robotic arm performs continuous object manipulation. Apply the **REINFORCE** algorithm to develop a policy that maximizes task completion accuracy. **.**

(5 Marks)

5. An autonomous vehicle is trained using **Q-Learning** and **Deep Q-Networks (DQN)**. Analyze the differences in learning performance, convergence speed, and scalability for complex driving environments. **.** **(5 Marks)**

6. A game-playing AI is implemented using **DQN**, **Double DQN (DDQN)**, and **Dueling DQN**. Analyze the advantages of DDQN and Dueling DQN over the standard DQN architecture. **. (5 Marks)**

7. A smart drone navigation system uses **A2C** and **A3C** algorithms. Analyze the impact of parallel learning on convergence speed, computational efficiency, and policy stability. **. (5 Marks)**

8. A healthcare decision-support system operates with incomplete patient information. Analyze how a Partially Observable Markov Decision Process (POMDP) improves decision-making compared to a standard MDP. **. (5 Marks)**

9. A robotic navigation system is trained using PPO, TRPO, and DDPG. Evaluate the suitability of each algorithm for continuous control tasks based on stability, sample efficiency, and computational complexity. **. (5 Marks)**

10. A smart city traffic management system uses **Multi-Agent Reinforcement Learning** (MARL) to coordinate traffic signals. Evaluate the **effectiveness of MARL** in improving traffic flow while considering scalability, coordination, fairness, and ethical challenges. **. (5 Marks)**

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively

ASSESSMENT – REINFORCEMENT LEARNING AND DYNAMIC PROGRAMMING

1. Dynamic Programming for Optimal Warehouse Robot Navigation

Marks: 5

Problem Analysis

A warehouse robot needs to transport goods from a starting location to a destination while avoiding obstacles and minimizing travel distance. The warehouse can be represented as a **grid-based Markov Decision Process (MDP)**.

- **States (S):** Robot positions in the warehouse.
- **Actions (A):** Up, Down, Left, Right.
- **Reward (R):**
  - +100 for reaching the destination.
  - -1 for every movement to encourage shorter paths.
  - -100 for hitting an obstacle.
- **Policy (π):** Determines the best action for each state.
- **Discount factor (γ):** Usually selected between 0 and 1.

Dynamic Programming Approach

The optimal value function can be calculated using the **Bellman optimality equation**:

V∗ (s)=max⁡a[R(s,a)+γ∑s′P(s′ ∣ s,a)V∗ (s′)]V^*(s)=\max_a\left[R(s,a)+\gamma\sum_{s'}P(s'|s,a)V^*(s')\right]

For a deterministic warehouse environment, this becomes:

V∗ (s)=max⁡a[R(s,a)+γV∗ (s′)]V^*(s)=\max_a[R(s,a)+\gamma V^*(s')]

The robot evaluates every possible action from each position and selects the action producing the highest expected future reward.

Value Iteration

Value iteration repeatedly updates the value of every state:

$$V_{k+1}(s)=\max_a[R(s,a)+\gamma V_k(s')]$$

The process continues until:

$$|V_{k+1}(s)-V_k(s)|<\theta$$

After convergence, the optimal policy is extracted using:

$$\pi^*(s)=\arg\max_a[R(s,a)+\gamma V^*(s')]$$

Analysis

Value iteration improves navigation efficiency because the robot does not simply search for a path once. It calculates the long-term value of different positions and actions. Obstacles receive poor rewards, while the destination receives a high reward. Therefore, the robot learns a route that balances **short distance, obstacle avoidance, and successful delivery**.

Example: If two routes exist, one short but close to obstacles and another slightly longer but safer, the reward structure can make the robot prefer the safer route.

Conclusion

Dynamic Programming and value iteration provide an optimal policy when the environment model is known. They enable the warehouse robot to systematically determine efficient and obstacle-free paths.

---

2. Monte Carlo Control for Customer Recommendation System

Marks: 5

Problem Analysis

A recommendation system interacts with customers over multiple sessions. The system must learn which products or content should be recommended based on previous customer interactions.

The problem can be modeled as an MDP:

- **State:** Customer preferences, browsing history, previous purchases.
- **Action:** Recommend a particular product/content.
- **Reward:** Click, purchase, rating, or engagement.
- **Episode:** One complete customer session.
- **Policy:** Strategy for selecting recommendations.

Monte Carlo Control Approach

Monte Carlo Control learns action values from **complete episodes** rather than requiring a known transition model.

For an episode:

S0,A0,R1,S1,A1,R2,…,STS_0,A_0,R_1,S_1,A_1,R_2,\ldots,S_T

The return from time tt is:

Gt=Rt+1+γRt+2+γ2Rt+3+⋯ G_t=R_{t+1}+\gamma R_{t+2}+\gamma^2R_{t+3}+\cdots

The action-value function is updated using:

Q(St,At)←Q(St,At)+α[Gt−Q(St,At)]Q(S_t,A_t)\leftarrow Q(S_t,A_t)+\alpha[G_t-Q(S_t,A_t)]

The policy is then improved using:

π(S)=arg⁡max⁡AQ(S,A)\pi(S)=\arg\max_A Q(S,A)

An **ε-greedy policy** can be used so that the system sometimes recommends new products rather than always choosing the currently best recommendation.

Working

1. Customer starts a session.
2. System recommends an item.
3. Customer interacts with the recommendation.
4. Rewards are collected throughout the session.
5. At the end of the session, the complete return is calculated.
6. Q(S,A)Q(S,A) is updated.
7. The recommendation policy is improved.
8. Future sessions use the updated policy.

Analysis

The major advantage of Monte Carlo Control is that the system learns directly from **actual customer experience**. For example, if a customer repeatedly clicks technology products after seeing a particular recommendation, the associated action receives a higher value.

Over many sessions, successful recommendations obtain higher QQ-values, while unsuccessful recommendations obtain lower values.

Conclusion

Monte Carlo Control enables the recommendation system to continuously improve from customer experience. It is particularly useful when complete interaction sessions are available and the system does not have an accurate model of customer behavior.

---

3. SARSA for an Uncertain Delivery Robot

Marks: 5

Problem Analysis

A delivery robot operates in an uncertain environment where actions may not always produce the expected result. For example, slippery floors, moving objects, pedestrians, or changing obstacles can affect navigation.

SARSA is an **on-policy temporal-difference reinforcement learning algorithm**.

The five elements represented by SARSA are:

(S,A,R,S′,A′)(S,A,R,S',A')

where:

- SS = Current state
- AA = Current action
- RR = Reward
- S′S' = Next state
- A′A' = Next action

SARSA Update Rule

The action-value function is updated using:

Q(S,A)←Q(S,A)+α[R+γQ(S′,A′)−Q(S,A)]Q(S,A)\leftarrow Q(S,A)+ \alpha[R+\gamma Q(S',A')-Q(S,A)]

The term:

R+γQ(S′,A′)R+\gamma Q(S',A')

represents the estimated target value.

Working

Suppose the robot is moving toward a delivery location.

1. Robot observes current state SS.
2. It selects action AA using an ε-greedy policy.
3. It receives reward RR.
4. It observes new state S′S'.
5. It selects the next action A′A'.
6. SARSA updates Q(S,A)Q(S,A).
7. The robot repeats the process.
8. After many experiences, the policy improves.

Adaptation to Environmental Changes

SARSA is particularly useful because it learns the value of actions according to the **policy that is actually being followed**.

If a previously safe route becomes dangerous, the robot begins receiving lower rewards when using that route. Consequently:

$Q(S,A)\downarrow$

for the unsafe action.

Alternative actions can then receive higher values, causing the robot to change its route.

Analysis

Compared with an offline fixed path, SARSA can continuously update its policy based on new environmental experiences. This makes it suitable for uncertain delivery environments.

For example:

Situation	**Reward**	**Effect on Q-value**
Successful delivery	+100	Increases
Normal movement	-1	Slight decrease
Collision	-100	Strong decrease
Delayed route	-10	Decreases

Conclusion

SARSA allows the delivery robot to learn safe and effective navigation through continuous interaction. Since it is an on-policy method, its learned behavior considers the consequences of the actions actually selected during exploration.

---

4. REINFORCE for Continuous Robotic Arm Manipulation

Marks: 5

Problem Analysis

A robotic arm performing object manipulation may have continuous states and actions such as:

- Joint angles
- Joint velocities
- Position
- Orientation
- Grip strength
- Movement direction

Traditional tabular methods become impractical for such high-dimensional problems. Therefore, **REINFORCE**, a policy-gradient algorithm, can be applied.

Policy Representation

The policy is represented by a parameterized function:

πθ(a ∣ s)\pi_\theta(a|s)

where:

- θ\theta = Policy parameters
- ss = Current state
- aa = Selected action

For continuous actions, the policy can generate parameters of a probability distribution such as a Gaussian distribution.

REINFORCE Algorithm

The objective is to maximize expected return:

J(θ)=Eπθ[G]J(\theta)=E_{\pi_\theta}[G]

The policy-gradient update is:

θ←θ+αGt∇θlog⁡πθ(At∣St)\theta \leftarrow \theta+ \alpha G_t\nabla_\theta \log \pi_\theta(A_t|S_t)

where:

Gt=∑k=tTγk−tRk+1G_t=\sum_{k=t}^{T}\gamma^{k-t}R_{k+1}

Working

1. Robotic arm observes the object and environment.
2. Policy network generates an action.
3. Arm moves and attempts manipulation.
4. Task performance generates rewards.
5. A complete trajectory is recorded.
6. Return GtG_t is calculated.
7. Policy parameters are updated.
8. Successful movements become more probable in future episodes.

Reward Design

A suitable reward function could be:

Event	**Reward**
Correct object grasp	+20
Successful placement	+100
Accurate positioning	+10
Excessive movement	-2
Dropping object	-50
Collision	-100

The policy therefore learns movements that maximize the probability of completing the manipulation task accurately.

Analysis

REINFORCE directly optimizes the policy instead of first constructing a complete action-value table. This makes it suitable for continuous and high-dimensional robotic tasks.

However, basic REINFORCE has **high variance** because updates depend on complete trajectories. Training can therefore be slower and less stable. Techniques such as baselines, advantage estimation, and actor-critic methods can improve performance.

Conclusion

REINFORCE enables the robotic arm to learn a probabilistic control policy directly from experience. With an appropriate reward function, successful manipulation behaviors become increasingly likely, improving task completion accuracy.

---

5. Q-Learning vs Deep Q-Networks for Autonomous Vehicle Navigation

Marks: 5

Problem Analysis

An autonomous vehicle must make decisions such as:

- Accelerate
- Brake
- Change lane
- Turn
- Maintain speed
- Avoid obstacles

The environment is highly complex, dynamic, and contains large amounts of sensory information. **Q-Learning** and **Deep Q-Networks (DQN)** can both learn action-selection policies, but their scalability is significantly different.

Q-Learning

Q-Learning is a model-free, off-policy reinforcement learning algorithm.

The update equation is:

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_{A'} Q(S',A') - Q(S,A)]$$

$$Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_{A'} Q(S',A') - Q(S,A)]$$

It normally stores $Q(S,A)$ in a table.

Advantages

- Simple implementation.
- Easy to understand.
- Fast for small state spaces.
- Does not require a model of the environment.
- Can converge to the optimal action-value function under suitable conditions.

Limitations

For autonomous vehicles, the state space can be extremely large.

For example:

$$State = (speed, position, lane, traffic, weather, distance, \ldots)$$

$$State = (speed, position, lane, traffic, weather, distance, \ldots)$$

A Q-table would become extremely large and inefficient.

Deep Q-Network (DQN)

DQN replaces the Q-table with a neural network:

$$Q(S,A;\theta)$$

$$Q(S,A;\theta)$$

The neural network takes the state as input and estimates Q-values for available actions.

For example:

Input:

Camera/sensor information → Neural Network → Q-values

[Q(s,a1),Q(s,a2),Q(s,a3),Q(s,a4)][Q(s,a_1),Q(s,a_2),Q(s,a_3),Q(s,a_4)]

The action with the highest Q-value is selected.

DQN commonly uses **experience replay** and a **target network** to improve training stability.

Comparative Analysis

Factor	**Q-Learning**	**DQN**
Representation	Q-table	Neural network
Small state spaces	Excellent	Usually unnecessary
Large state spaces	Poor scalability	Much better
Image/sensor input	Not suitable directly	Suitable with appropriate architecture
Memory requirement	Can become very large	Neural-network parameters
Training complexity	Low	High
Convergence	Often fast for small problems	Can require substantial training
Generalization	Limited	Better
Complex driving environment	Poor	More suitable
Continuous/high-dimensional observations	Difficult	Better suited, although vanilla DQN still requires discrete actions

Learning Performance

For a small simulated road environment, traditional Q-Learning may learn faster because the number of states and actions is small.

However, as the environment becomes more complex, Q-Learning requires a huge table. DQN uses a neural network to approximate the Q-function, allowing it to handle much larger state representations.

Convergence Speed

Q-Learning can converge relatively quickly in small environments because it directly updates table entries.

DQN may initially converge more slowly because neural-network training requires many experiences. However, techniques such as **experience replay** and **target networks** make learning more stable and scalable.

Scalability

This is the biggest difference.

A Q-table may require:

$|S| \times |A|$

entries.

If the number of states becomes millions or billions, storing and learning individual entries becomes impractical.

DQN instead learns a function:

$Q(s,a;\theta)$

and can generalize across similar states. Therefore, DQN is considerably more suitable for complex autonomous-driving environments.

Overall Evaluation

Criterion	**Best Method**
Simple environment	Q-Learning

Criterion	Best Method
Easy implementation	Q-Learning
Small discrete state space	Q-Learning
Large state space	DQN
Complex sensor observations	DQN
Generalization	DQN
Computational simplicity	Q-Learning
Autonomous driving scalability	DQN

Conclusion

Q-Learning is appropriate for small and discrete autonomous-driving simulations, where the state space can be represented using a manageable Q-table. **DQN is more suitable for complex driving environments** because it uses deep neural networks to approximate the Q-function and can generalize across large state spaces.

Therefore, as the autonomous vehicle environment becomes more realistic—with complex traffic, sensor inputs, dynamic obstacles, and numerous possible states—**DQN provides better scalability and practical learning capability**, although it requires greater computational resources and careful training.

Overall Assessment Summary

Question	Technique	Main Learning Principle	Suitable Application
1	Dynamic Programming / Value Iteration	Bellman optimality	Warehouse navigation
2	Monte Carlo Control	Learning from complete episodes	Recommendation systems
3	SARSA	On-policy TD learning	Uncertain delivery robot
4	REINFORCE	Policy-gradient	Robotic arm

Question	Technique	Main Learning Principle	Suitable Application
		optimization	manipulation
5	Q-Learning vs DQN	Value-based deep RL	Autonomous vehicles

6. Analysis of DQN, Double DQN (DDQN), and Dueling DQN

Marks: 5

Problem Analysis

A game-playing AI must select the best action based on the current game state. **Deep Q-Network (DQN)** uses a neural network to estimate the action-value function:

Q(s,a;θ)Q(s,a;\theta)

The standard DQN selects an action using:

a∗ =arg⁡max⁡aQ(s,a)a^*=\arg\max_a Q(s,a)

However, standard DQN has limitations such as **overestimation of Q-values** and inefficient learning when many actions have similar values.

Two improvements are **Double DQN (DDQN)** and **Dueling DQN**.

1. Standard DQN

DQN uses one neural network to estimate Q-values and a target network for stable target calculation.

The target is:

Y=r+γmax⁡a′Qtarget(s′,a′)Y=r+\gamma\max_{a'}Q_{target}(s',a')

The maximum Q-value is used both for selecting and evaluating the next action, which can result in overestimation.

---

2. Double DQN

DDQN separates **action selection** from **action evaluation**.

The target is:

$$Y=r+\gamma Q_{target}\left(s', \arg\max_{a'}Q_{online}(s',a')\right)$$

Here:

- Online network → selects the best action.
- Target network → evaluates that selected action.

Advantage of DDQN

The separation reduces the tendency of standard DQN to overestimate action values.

Feature	**DQN**	**DDQN**
Action selection	Target network	Online network
Action evaluation	Target network	Target network
Q-value overestimation	Higher	Lower
Learning stability	Moderate	Better
Policy quality	Good	Usually improved

Analysis: DDQN is particularly beneficial in games where several actions have similar values because it prevents the network from repeatedly assigning unrealistically high values to certain actions.

---

3. Dueling DQN

Dueling DQN divides the network into two streams:

V(s)V(s)

and

A(s,a)A(s,a)

where:

- V(s)V(s) = Value of being in state ss.
- A(s,a)A(s,a) = Advantage of taking action aa.

They are combined approximately as:

Q(s,a)=V(s)+A(s,a)Q(s,a)=V(s)+A(s,a)

Advantage of Dueling DQN

Dueling DQN can learn **which states are valuable even when the choice of action has little effect**.

For example, in a game, a player may be in a safe position where moving left, right, or staying has nearly the same consequence. Standard DQN must estimate every action separately, whereas Dueling DQN can first learn that the state itself is valuable.

Overall Analysis

DDQN primarily addresses **Q-value overestimation**, while Dueling DQN improves **state-value and action-advantage representation**.

Therefore:

DDQN → More accurate Q-values\boxed{\text{DDQN → More accurate Q-values}} Dueling DQN → Better state/action representation\boxed{\text{Dueling DQN → Better state/action representation}}

Conclusion

DDQN and Dueling DQN improve standard DQN in different ways. DDQN increases learning reliability by reducing overestimation, while Dueling DQN improves learning efficiency by separating state value from action advantage. Combining both ideas can provide a stronger architecture for complex game-playing AI.

---

7. Analysis of A2C and A3C for Smart Drone Navigation

Marks: 5

Problem Analysis

A smart drone operates in a continuously changing environment. It must learn actions such as:

- Move forward
- Turn
- Increase altitude
- Decrease altitude
- Avoid obstacles
- Reach destination

A2C (Advantage Actor-Critic) and **A3C (Asynchronous Advantage Actor-Critic)** use both policy and value functions.

A2C

A2C is a synchronous actor-critic algorithm.

It consists of:

- **Actor:** Determines the action policy.
- **Critic:** Estimates the value of the current state.

The advantage can be calculated as:

A(s,a)=Q(s,a)−V(s)A(s,a)=Q(s,a)-V(s)

A2C collects experiences from multiple environments and performs updates synchronously.

A3C

A3C uses multiple worker agents operating in parallel.

Each worker:

1. Interacts with its own environment.
2. Collects experience.
3. Calculates gradients.
4. Updates shared model parameters asynchronously.

Conceptually:

Worker1→Worker_1 \rightarrow Worker2→Global NetworkWorker_2 \rightarrow Global\ Network Worker3→Worker_3 \rightarrow Workern→Worker_n \rightarrow

---

Impact of Parallel Learning

1. Convergence Speed

A3C can collect experiences from multiple environments simultaneously.

Therefore:

Experience Collection Rate↑\text{Experience Collection Rate} \uparrow

This can reduce wall-clock training time compared with a single-agent approach.

A2C also benefits from parallel environments, but workers typically synchronize before an update.

2. Computational Efficiency

A3C can make effective use of multiple CPU cores because workers operate independently.

However, asynchronous updates introduce coordination and synchronization overhead.

A2C is often easier to implement efficiently on modern hardware because batches can be processed together.

3. Policy Stability

A2C performs synchronized updates, so the collected experiences are more consistent when the policy is updated.

A3C workers may generate experiences using slightly different versions of the policy. This can increase gradient diversity and exploration, but asynchronous updates may also introduce noise.

Factor	**A2C**	**A3C**
Learning	Synchronous	Asynchronous
Experience collection	Parallel	Parallel
Convergence	Stable	Often fast
Implementation	Simpler	More complex
CPU utilization	Good	Excellent
Policy stability	Generally high	Can be noisier
Exploration	Moderate	Strong due to diverse workers

Analysis

For drone navigation, A3C can explore different environmental conditions simultaneously—for example, different obstacle configurations and weather conditions. This can improve learning speed.

A2C, however, provides more synchronized and predictable updates, which may be advantageous when stable training is important.

Conclusion

A3C provides strong parallelism and diverse exploration, potentially improving convergence speed. A2C provides simpler and more synchronized learning. For large-scale drone simulation, **A3C is advantageous for parallel exploration**, while **A2C may offer simpler and more stable implementation**.

---

8. POMDP for Healthcare Decision Support with Incomplete Information

Marks: 5

Problem Analysis

A healthcare decision-support system often does not have complete information about a patient. For example:

- Some symptoms may not be reported.
- Test results may be unavailable.
- Patient history may be incomplete.
- The patient's actual health condition cannot always be directly observed.

A standard **Markov Decision Process (MDP)** assumes that the current state is sufficiently observable.

A **Partially Observable Markov Decision Process (POMDP)** is more appropriate when the actual state cannot be directly observed.

Standard MDP

An MDP can be represented as:

(S,A,P,R,γ)(S,A,P,R,\gamma)

where:

- SS = States
- AA = Actions
- PP = Transition probabilities
- RR = Rewards
- γ\gamma = Discount factor

The agent assumes it knows the current state.

For example:

s=patient has condition Xs=\text{patient has condition X}

The system chooses an appropriate treatment.

POMDP

A POMDP extends the MDP by adding observations:

(S,A,P,R,O,Z,γ)(S,A,P,R,O,Z,\gamma)

where:

- OO = Observations
- ZZ = Observation probabilities

The actual patient condition may be hidden, while the system receives incomplete observations.

Instead of directly knowing:

P(s)P(s)

the system maintains a **belief state**:

b(s)=P(s ∣ history)b(s)=P(s|history)

The belief state represents the probability that the patient is in each possible health condition.

Example

Suppose a patient has:

- Fever
- Fatigue
- Incomplete test results

The system cannot be certain about the patient's actual condition.

It may estimate:

Possible condition	**Belief**
Condition A	0.60
Condition B	0.30
Condition C	0.10

Based on these probabilities, the system can select an appropriate next action, such as:

- Request additional test.
- Monitor the patient.
- Recommend clinical review.

MDP vs POMDP

Feature	**MDP**	**POMDP**
State information	Fully observable	Partially observable
Uncertainty	Lower	Higher

Feature	MDP	POMDP
Patient history	Direct state representation	Included through observations/history
Decision basis	Current state	Belief state
Suitable for incomplete medical data	Limited	Highly suitable
Decision quality under uncertainty	Lower	Better

Analysis

The major advantage of POMDP is that it does not force the healthcare system to assume that incomplete information is complete. It explicitly models uncertainty.

For example, instead of saying:

> "The patient definitely has condition A."

the system can reason:

$$P(A)=0.60, P(B)=0.30, P(C)=0.10$$

It can then choose actions that maximize expected benefit while considering uncertainty.

Ethical Consideration

Healthcare decisions are high-stakes. A POMDP should therefore support—not replace—qualified clinical judgment. The system should also account for patient privacy, bias, explainability, and uncertainty.

Conclusion

POMDP provides a more realistic framework for healthcare decision support because it models hidden patient states and incomplete observations. By maintaining belief states, the system can make more informed decisions under uncertainty than a standard fully observable MDP.

---

9. Evaluation of PPO, TRPO, and DDPG for Continuous Robotic Navigation

Marks: 5

Problem Analysis

A robotic navigation system may require continuous actions such as:

- Steering angle
- Wheel velocity
- Motor power
- Turning rate

Therefore, the algorithm must handle continuous control efficiently.

The three algorithms are:

1. **PPO – Proximal Policy Optimization**
2. **TRPO – Trust Region Policy Optimization**
3. **DDPG – Deep Deterministic Policy Gradient**

---

1. PPO

PPO is a policy-gradient algorithm designed to maintain stable updates.

Its clipped objective can be represented as:

LCLIP(θ)=E[min⁡(rt(θ)At,clip(rt(θ),1−ϵ,1+ϵ)At)]L^{CLIP}(\theta)= E[\min(r_t(\theta)A_t, clip(r_t(\theta),1-\epsilon,1+\epsilon)A_t)]

where rtr_t is the probability ratio between new and old policies.

Advantages

- Good training stability.
- Easier implementation than TRPO.
- Works well for continuous control.
- Reuses collected samples for multiple optimization epochs.
- Strong practical performance.

Limitation

It can still require many environment interactions and computational resources.

---

2. TRPO

TRPO restricts policy updates so that the new policy does not move too far from the old policy.

It approximately solves:

max⁡θL(θ)\max_\theta L(\theta)

subject to:

DKL(πold∣∣πθ)≤δD_{KL}(\pi_{old}||\pi_\theta)\leq\delta

Advantages

- Strong policy stability.
- Prevents excessively large policy updates.
- Good theoretical foundation.

Limitations

- Computationally expensive.
- More complicated implementation.
- Constrained optimization increases computational overhead.

---

3. DDPG

DDPG is an actor-critic algorithm designed specifically for continuous action spaces.

It contains:

- **Actor:** Produces continuous action.
- **Critic:** Estimates Q(s,a)Q(s,a).

The actor learns:

a=μθ(s)a=\mu_\theta(s)

The critic estimates:

Q(s,a)Q(s,a)

Advantages

- Naturally supports continuous actions.
- Can learn deterministic policies.
- Experience replay improves data usage.
- Suitable for high-dimensional control.

Limitations

- Can be sensitive to hyperparameters.
- Training may become unstable.
- Exploration can be challenging.

Comparative Evaluation

Criterion	**PPO**	**TRPO**	**DDPG**
Continuous control	Excellent	Excellent	Excellent

Criterion	PPO	TRPO	DDPG
Stability	High	Very high	Moderate
Sample efficiency	Good	Moderate	Good
Computational complexity	Moderate	High	Moderate
Implementation complexity	Moderate	High	Moderate
Policy type	Stochastic	Stochastic	Deterministic
Training reliability	High	High	Moderate
Suitable for robotic navigation	Very suitable	Suitable	Suitable

Analysis

PPO provides the best overall balance between stability, performance, and computational complexity.

TRPO provides strong theoretical stability but requires more complex optimization and computational resources.

DDPG is attractive when precise continuous actions are required, but its training stability is generally more sensitive.

Final Evaluation

For a robotic navigation system:

PPO = Best overall practical choice\boxed{\text{PPO = Best overall practical choice}} TRPO = Strong stability, high computational cost\boxed{\text{TRPO = Strong stability, high computational cost}} DDPG = Strong continuous-action capability, but less stable\boxed{\text{DDPG = Strong continuous-action capability, but less stable}}

Conclusion

PPO is generally the most suitable practical choice for continuous robotic navigation because it provides a good balance between stability, sample utilization, and computational

complexity. TRPO is useful where strong policy-update constraints are important, while DDPG is useful for deterministic continuous control but requires careful tuning.

---

10. Multi-Agent Reinforcement Learning for Smart City Traffic Management

Marks: 5

Problem Analysis

A smart city may contain hundreds or thousands of traffic signals. Instead of controlling each traffic signal independently, **Multi-Agent Reinforcement Learning (MARL)** allows multiple agents to learn and coordinate.

Each traffic signal can be considered an agent.

Example

Agent1→Signal1Agent_1 \rightarrow Signal_1 Agent2→Signal2Agent_2 \rightarrow Signal_2 Agent3→Signal3Agent_3 \rightarrow Signal_3

Each agent observes traffic conditions and chooses actions such as:

- Extend green light.
- Reduce green-light duration.
- Switch phase.
- Prioritize emergency vehicles.

MARL Model

Each agent has:

State

si=(queue length, waiting time, traffic density, speed)s_i=(queue\ length,\ waiting\ time,\ traffic\ density,\ speed)

Actions

$A_i = \{\text{change phase}, \text{maintain phase}, \text{extend green}\}$

Reward

A possible reward is:

$$R_i = -(\alpha Q_i + \beta W_i + \gamma D_i)$$

where:

- Q_i = Queue length
- W_i = Waiting time
- D_i = Traffic delay

The objective is to minimize congestion and improve traffic flow.

Effectiveness of MARL

1. Traffic Flow Improvement

MARL allows neighboring traffic signals to coordinate.

For example, if one intersection experiences heavy traffic, nearby signals can adjust their timing to prevent traffic from accumulating further.

This can reduce:

- Average waiting time
- Queue length
- Travel time
- Traffic congestion

A coordinated system can create smoother traffic progression across multiple intersections.

---

2. Scalability

A major challenge is the number of agents.

For NN traffic signals:

Atotal=A1×A2×⋯ ×ANA_{total}=A_1\times A_2\times\cdots\times A_N

The joint action space can grow rapidly.

This is known as the **curse of dimensionality**.

Solution

MARL can use:

- Decentralized execution.
- Parameter sharing.
- Local observations.
- Hierarchical control.
- Centralized training with decentralized execution.

These techniques can make large traffic networks more manageable.

---

3. Coordination

Independent agents may learn conflicting behaviors.

For example:

- Signal A prioritizes its own traffic.
- Signal B also prioritizes its own traffic.
- Their decisions together may increase congestion.

MARL solves this by incorporating cooperation into the learning objective.

A shared reward can encourage agents to optimize overall traffic flow rather than only local performance.

---

4. Fairness

Traffic optimization should not focus only on major roads.

If the system always prioritizes roads with heavy traffic, vehicles on smaller roads may experience excessive waiting times.

Therefore, the reward function should consider:

Fairness=balanced waiting times across traffic flowsFairness = \text{balanced waiting times across traffic flows}

Emergency vehicles, public transportation, pedestrians, and cyclists may also require appropriate priority.

---

5. Ethical Challenges

Smart traffic systems affect real people, so ethical considerations are important.

Privacy

Traffic cameras and sensors may collect sensitive information. Data collection should be minimized and protected.

Bias

If training data favors particular roads or traffic patterns, the learned system may systematically disadvantage certain areas.

Safety

The system must never prioritize traffic efficiency at the expense of road safety.

Transparency

Traffic authorities should be able to understand why significant traffic-control decisions are being made.

Emergency Priority

Emergency vehicles may require priority, but this should be designed so that other road users are not exposed to unsafe conditions.

MARL Evaluation

Criterion	**Evaluation**
Traffic-flow improvement	High potential
Coordination	Excellent compared with independent control
Scalability	Challenging but manageable
Adaptability	High
Fairness	Requires explicit reward design
Safety	Must be strongly constrained
Privacy	Requires secure data handling
Computational requirements	High
Real-world deployment	Requires extensive testing

Overall Analysis

MARL is more effective than treating every traffic signal as an independent system because agents can learn coordinated strategies. However, increasing the number of intersections increases the complexity of joint decision-making.

A practical architecture can therefore use **centralized training and decentralized execution (CTDE)**. During training, agents can use global information to learn coordination, while during operation each signal can make decisions using local information.

Conclusion

MARL can significantly improve smart-city traffic management by enabling traffic signals to coordinate dynamically according to real-time traffic conditions. However, successful deployment requires solving scalability and coordination problems while explicitly addressing **fairness, safety, privacy, transparency, and ethical decision-making**. Therefore, MARL is a powerful approach, but it should be deployed with strong safety constraints and human oversight.